

# Model Evaluation, Cross Validation, PCA & Feature Importance

Sattam Altwaim

[sattam.tuuaim@kaust.edu.sa](mailto:sattam.tuuaim@kaust.edu.sa)



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology



KAUST Academy & Mawhiba  
AI Olympiad Training (Beginners)  
Week 2, Day 13, Self-study, Olympiad practice

1 October 2026

1. Where This Fits
2. Learning Outcomes
3. Introduction
4. Choosing the Folds
5. Common Tricks
6. Common Mistakes
7. Tips
8. Common Practices
9. Olympiad Practice
10. Check Yourself
11. Limitations and What Comes Next
12. Summary
13. References

- ▶ **This week gave you the models:** lines and logistic regression (Day 08), the metrics, the overfitting alarm and regularization (Day 09), KNN, SVMs, trees, forests, boosting and tuning (Day 10), each deepened on Days 11 and 12.
- ▶ **This deck is about the number you trust.** Every model choice this week was made by comparing a validation score. The score is only as good as the way it was produced, and real solutions get that wrong more often than they get the model wrong: the organisers' own sample solution for the HAIO admissions task submits 0/1 labels to a ROC-AUC judge.
- ▶ **How to use it:** one move per frame: the situation, then the action, with code where the action is code. Every number comes from running that code on the admissions, credit-score and ECG tables. Keep the routine frame open on Friday.

## Four real cases

- ▶ **OJIA 2025 Credit Score:** the same customer appears in up to 8 monthly rows. A random 5-fold split says accuracy 0.754; folds that keep each customer on one side say **0.689**. Six points of the score were the model recognising customers, not predicting credit.
- ▶ **HAIO 2025 admissions:** a table shipped sorted by the target breaks plain KFold: three of five folds contain no admitted student and ROC-AUC is undefined.
- ▶ **JOAI 2025:** the public leaderboard used half of the test set; the write-up that trusted its cross-validation rose about 30 places on the private half.
- ▶ **Polish OAI 2025 ECG:** the same four features and the same tiny forest scored balanced accuracy 83% (35 points) before and 97% (97 points) after the authors read the confusion matrix and fixed the preprocessing.

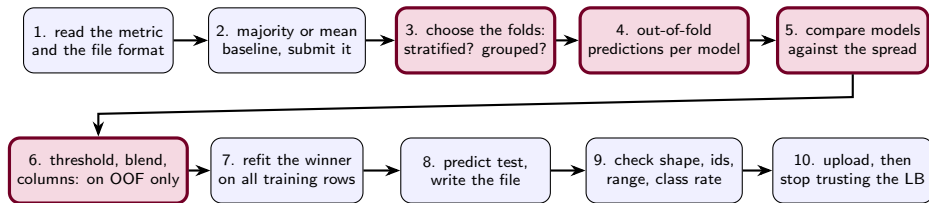
## After this deck you can

Produce a validation number that predicts the judge's number and know how much to trust it, pick a threshold or a blend without touching the test set, see a 150-column table in two dimensions, tell real importance from the fake kind, and run the submission routine without a surprise.

By the end of this deck, you will be able to:

- ▶ **Choose** the cross-validation scheme a table needs (stratified, shuffled, grouped) and explain what each one protects against.
- ▶ **Use** out-of-fold predictions to pick a threshold, blend models and diagnose errors without touching the test set.
- ▶ **Read** the spread of a cross-validated score and decide whether a difference between two models is real.
- ▶ **Apply** PCA to look at high-dimensional data and say when it helps and when it hurts a model.
- ▶ **Compare** impurity and permutation importance, spot the random-column trap, and prune columns only when the score agrees.
- ▶ **Run** the submission routine: metric, baseline, CV, refit on all rows, file checks.

# Model Evaluation: **Introduction**



- Steps 1, 2, 8 and 9 were Day 06; steps 7 and 8 were every lab this week. The purple steps are today's: they decide whether the number in step 5 means anything.
- The routine is the same for a regression, a binary and a multiclass task; only the metric and the kind of folds change.

| Number                  |                 | How                                                                      | Good for                                                                   |
|-------------------------|-----------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------|
| one                     | hold-out split  | <code>train_test_split</code> , score once                               | quick checks while building features; cheap                                |
| k-fold                  | mean and spread | <code>cross_val_score(cv=5)</code>                                       | comparing models and settings; knowing the noise                           |
| out-of-fold predictions | pre-dictions    | <code>cross_val_predict</code> with <code>method= "predict_proba"</code> | one honest prediction per training row: thresholds, blends, error analysis |

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
scores = cross_val_score(model, X, y, cv=cv, scoring="roc_auc")    # 5 numbers
oof = cross_val_predict(model, X, y, cv=cv, method="predict_proba") # per row
print(scores.mean(), scores.std(), roc_auc_score(y, oof[:, 1]))
```

- Every row's out-of-fold probability comes from a model that never saw that row: the OOF table is the closest thing you have to the hidden test set, and you may do anything with it. None of the three can be computed on the test rows; those belong to the judge.

# Model Evaluation: **Choosing the Folds**

## Definition

`KFold(k)` cuts the row *positions*  $0, 1, \dots, n - 1$  into  $k$  consecutive blocks; each block is the validation set once. It never looks at the labels. `shuffle=True` permutes the positions first.

|                                         |   |   |   |   |   |   |   |   |   |    |
|-----------------------------------------|---|---|---|---|---|---|---|---|---|----|
| ten rows, sorted by label:              | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| <code>KFold(2)</code> fold of each row: | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2  |
| <code>KFold(2, shuffle=True)</code> :   | 2 | 1 | 1 | 2 | 1 | 2 | 2 | 2 | 1 | 1  |

Purple rows are the three positives. Without shuffling, fold 1 holds rows 1 to 5 (all three positives) and fold 2 rows 6 to 10: **no positive at all**, so recall, F1 and ROC-AUC are undefined there. Shuffling helps on average (here 2 and 1) but is luck: another seed can give 0 or 3. Plain `KFold(k, shuffle=True)` is the splitter for **regression**, where there are no classes to balance.

## Definition

`StratifiedKFold(k)` splits *each class separately* into  $k$  nearly equal parts and builds fold  $j$  from part  $j$  of every class: every fold keeps the table's class shares.

|                     |   |   |   |   |   |   |   |   |   |    |
|---------------------|---|---|---|---|---|---|---|---|---|----|
| the same ten rows:  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| StratifiedKFold(2): | 1 | 1 | 2 | 1 | 1 | 1 | 2 | 2 | 2 | 2  |

## The mechanism by hand

Positives, 3 rows into 2 parts: 2 + 1 (rows 1, 2 to fold 1; row 3 to fold 2). Negatives, 7 rows: 3 + 4. Fold 1 = {1, 2, 4, 5, 6} with 2 positives of 5, fold 2 = {3, 7, 8, 9, 10} with 1 of 5: as close to the table's 30% as 10 rows allow. With 6,885 admissions rows all five folds got 29.8%.

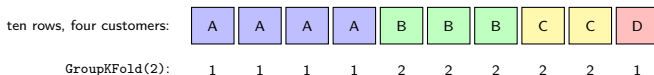
```
cv = StratifiedKFold(5, shuffle=True, random_state=0)    # shuffle inside each class,
    then split
scores = cross_val_score(model, X, y, cv=cv, scoring="roc_auc")
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.2, stratify=y)  # the hold-
    out version
```

| Scenario                                                                          | What goes wrong without it                                                                          | What to do                                                                                         |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| imbalanced yes/no target (admissions: 29.8% yes; a disease at 1%)                 | a fold with few positives makes recall and F1 jump; at 1% a fold can hold no positive               | <code>StratifiedKFold(5, shuffle=True); stratify=y</code> in the hold-out too                      |
| several classes, some rare (ECG: 5 classes, 150 beats per disorder among 2,000)   | a plain fold can miss a class, so macro F1 and balanced accuracy go undefined or noisy              | <code>stratify</code> ; every class is balanced, not only the rare one                             |
| a small table (300 admissions rows)                                               | the share of "yes" swung from 18% to 40% between folds over 20 draws; stratified: 33% in every fold | <code>stratify</code> , and repeat with several <code>random_state</code> values                   |
| a file sorted or grouped by class                                                 | consecutive folds hold one class each (the <code>nan</code> ROC-AUC of the mistake frame)           | <code>stratify</code> with <code>shuffle=True</code>                                               |
| regression with a skewed target (Steam: most games under 20 dollars, a few at 60) | the few expensive games land in one fold and its MAE is off by itself                               | bin the target with <code>pd.qcut(y, 5, labels=False)</code> and <code>stratify</code> on the bins |

The ECG file is shuffled and the HAIIO file was not sorted: the sorted case appears only on files the organisers ordered, and you learn which by checking `y.head(20)` and `y.tail(20)`.

## Definition

GroupKFold(k) takes a **group id per row** (a customer, a patient, a device) and puts all rows of a group into one fold, so no group is on both sides of a split. The score then measures the model on *groups it has never seen*.



## The mechanism by hand

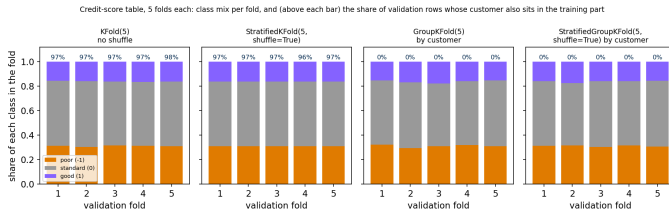
Groups by size: A (4 rows), B (3), C (2), D (1), each sent to the fold with the fewest rows so far: A to fold 1, B to fold 2, C to fold 2, D to fold 1. Fold 1 = {A, D}, fold 2 = {B, C}; A is never split, but all three positives sit in fold 1.

```
scores = cross_val_score(model, X, y, cv=GroupKFold(5), groups=train["Customer_ID"])  
cv = StratifiedGroupKFold(5, shuffle=True, random_state=0) # groups together AND  
    classes balanced
```

| Scenario                                                                                                  | Group column              | What a random split leaks                                                     | Which split mirrors the test?                                              |
|-----------------------------------------------------------------------------------------------------------|---------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| a customer's monthly rows<br>(credit score: up to 8 rows<br>per customer, 59% keep one<br>score all year) | Customer_ID               | January trains, February val-<br>idates: the model recognises<br>the customer | test customers also in train<br>(99%): random; new cus-<br>tomers: grouped |
| a patient's visits or scans                                                                               | patient id                | the same patient's other visit                                                | almost always new patients:<br>grouped                                     |
| photos of the same person,<br>product or scene                                                            | person or prod-<br>uct id | a near-duplicate image                                                        | new people: grouped                                                        |
| readings from one sensor or<br>device                                                                     | device id                 | the device's own calibration                                                  | new devices: grouped                                                       |
| texts by the same author or<br>from the same source                                                       | author id                 | style, not content                                                            | read the task statement                                                    |
| rows ordered in time, test<br>rows later                                                                  | the date                  | the future predicts the past                                                  | TimeSeriesSplit: train be-<br>fore, validate after                         |

The question is always the same: **does the test set contain the same groups as the training set, or new ones?** One line answers it: `test[group].isin(train[group]).mean()`. No id column does not mean no groups: 150 heartbeats from one recording, 20 frames from one video. Ask where the rows came from.

# The Real Table, Four Ways



Source: OJIA 2025 credit-score training table (44,207 rows, 11,027 customers, three classes); `scripts/day13_folds-figure.py`.

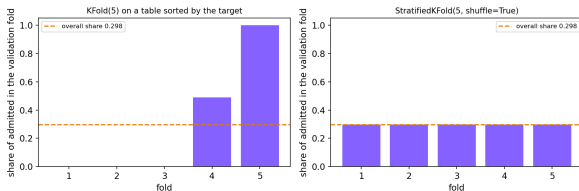
- ▶ Class mix: all four keep it near 31/53/16, because this file is not sorted; stratification is insurance here, not a gain.
- ▶ Leakage: under the two plain splitters 97% of validation rows have their customer in the training part; under the two group splitters 0%. That is the whole 0.754 against 0.689 of the mistake frame.
- ▶ StratifiedGroupKFold keeps both properties: the default whenever a group column exists and the target has classes.

| Target         | Rows             | Splitter                              | Single hold-out              |
|----------------|------------------|---------------------------------------|------------------------------|
| classes        | independent      | StratifiedKFold(5, shuffle=True)      | train_test_split(stratify=y) |
| number         | independent      | KFold(5, shuffle=True)                | train_test_split             |
| number, skewed | independent      | StratifiedKFold on pd.qcut bins       | stratify=bins                |
| classes        | share a group id | StratifiedGroupKFold(5, shuffle=True) | GroupShuffleSplit            |
| number         | share a group id | GroupKFold(5)                         | GroupShuffleSplit            |
| any            | ordered in time  | TimeSeriesSplit(5)                    | the last 20% of the dates    |

A fixed `random_state` wherever there is a `shuffle`, so the folds are the same for every model you compare. One check overrides the table: **mirror the test**. Test rows that share groups with the training rows make a grouped split pessimistic; new groups make a random split a lie. The credit-score table was the first case.

# Model Evaluation: **Common Tricks**

# Trick: Stratified, Shuffled Folds for Any Classification



Source: HAO 2025 admissions data sorted by the target, two splitters;  
scripts/day13\_evaluation\_figures.py.

## When it applies

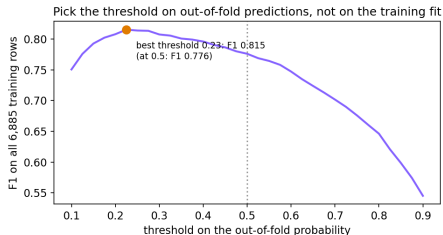
Every classification task, and especially an imbalanced one (29.8% admitted) or a table that arrives sorted or grouped by class.

## What to do

`StratifiedKFold(5, shuffle=True, random_state=0)`: every fold gets the same class share (0.298 in all five). Plain `KFold` on the sorted table produced folds with 0% and 100% admitted and three ROC-AUC values of `nan`.

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
```

# Trick: Pick the Threshold on Out-of-Fold Predictions



Source: admissions data, logistic regression, 5-fold out-of-fold probabilities;  
scripts/day13\_evaluation\_figures.py.

## When it applies

The metric is F1, balanced accuracy or recall, which depend on the 0.5 cut that predict uses.

## What to do

Compute the metric for 30 thresholds on the OOF probabilities and keep the best: F1 0.776 at 0.5, **0.815 at 0.23**. Apply that cut to the test probabilities.

```
ths = np.linspace(0.1, 0.9, 33)
f1s = [f1_score(y, oof >= t) for t in ths]
best_t = ths[int(np.argmax(f1s))]
labels = (pipe.predict_proba(X_test)[: ,
1]
>= best_t).astype(int)
```

# Trick: Blend Two Different Models

## When it applies

Two models with similar scores that make different mistakes: a linear model and a tree ensemble, or the five models of a cross-validation.

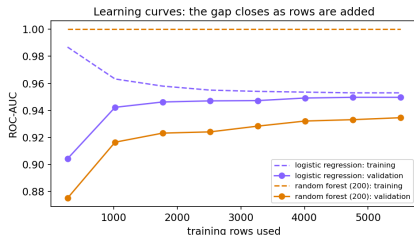
## What to do

Average their probabilities (or predictions). On the admissions OOF table: logistic 0.9496, boosting 0.9464, **mean of the two 0.9510**, better than either. The JOAI 2025 gold notebook trains one model per fold and averages the five on the test set: public 0.984, private 0.981, 10th overall.

```
oof_l = cross_val_predict(logit, X, y, cv=cv, method="predict_proba")[:, 1]
oof_g = cross_val_predict(boost, X, y, cv=cv, method="predict_proba")[:, 1]
print(roc_auc_score(y, oof_l), roc_auc_score(y, oof_g), roc_auc_score(y, (
    oof_l + oof_g) / 2))
test_blend = (logit.fit(X, y).predict_proba(X_test)[:, 1] + boost.fit(X, y).
    predict_proba(X_test)[:, 1]) / 2
```

Judge the blend on OOF, never by fitting weights on the test file; two or three models are enough. Regression: average predictions. ROC-AUC: average probabilities. Accuracy or F1: average probabilities, then cut at the OOF-chosen threshold.

# Trick: Draw the Learning Curve Before Hunting for Data



Source: admissions data, `learning_curve` with 5 stratified folds;  
`scripts/day13_evaluation_figures.py`.

## When it applies

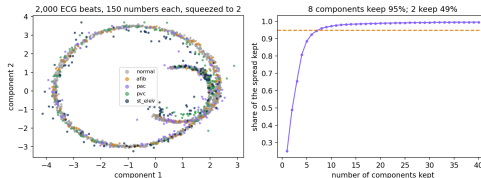
You wonder whether to spend the hour on more data or on a better model.

## What to do

`learning_curve(model, X, y, train_sizes=...)`: the validation score against the number of training rows. Logistic regression is flat from 2,500 rows on: more rows buy nothing. The forest still rises at 5,508 rows: more rows (or more regularization) would help it.

The gap between the dashed training curve and the solid validation curve is Day 09's alarm, drawn against data size instead of model size.

# Trick: PCA to See 150 Columns at Once



Source: II Olimpiada AI 2025 ECG data, 2,000 training beats of 150 samples, PCA fitted on the training rows; `scripts/day13_evaluation_figures.py`.

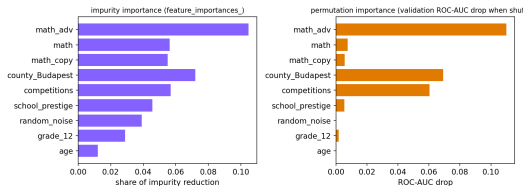
## When it applies

Too many columns to plot (sensors, pixels, the 150 voltages of a beat); you want to see whether the classes separate.

## What to do

`PCA(2).fit(X_train)` finds the two directions of largest spread (Day 08); plot the rows on them, coloured by class. Two components keep 49%, eight keep 95%: a beat is nearly an 8-number object. The classes overlap, honestly: PCA never looks at the labels.

# Trick: Permutation Importance, Not Only feature\_importances\_



Source: admissions data plus a uniform random column and a copy of `math`, random forest of 300 trees, `permutation_importance` on the validation rows; `scripts/day13.evaluation_figures.py`.

## What to do

`permutation_importance(model, X_val, y_val, scoring="roc_auc", n_repeats=10)` shuffles one column at a time and measures how much the validation score drops. A column the model does not need drops nothing. Rank columns by that, explain the model with that, select columns with that.

Left: the random column ranks 8th of 56 by impurity (0.039); a forest always finds a split in noise. Right: its permutation importance is 0.0002. The copy of `math` shows the other trap: identical columns share one importance in halves.

# Model Evaluation: **Common Mistakes**

# Mistake: Rows That Share a Group, Split at Random

## The situation

OJIA 2025 Credit Score: 44,207 rows but only 11,027 customers; each customer has up to 8 monthly rows, and 59% of customers keep the same score in every month. A random split puts January on the training side and February on the validation side, and the model “predicts” the customer it has already seen.

| HistGradientBoosting, 5 folds | random KFold | GroupKFold by customer |
|-------------------------------|--------------|------------------------|
| accuracy                      | 0.754        | <b>0.689</b>           |

```
from sklearn.model_selection import GroupKFold
scores = cross_val_score(model, X, y, cv=GroupKFold(5), groups=train["
    Customer_ID"])
print(test["Customer_ID"].isin(train["Customer_ID"]).mean())      # 0.99: the
    test customers are in train too
```

- **Which number is right?** It depends on the test set. Here 6,247 of the 6,291 test customers also appear in train, so the random number is the one the judge reproduces (the reference solution scored 0.79). New test customers would make only the grouped number survive. Check the overlap in one line, then choose the folds that mirror the test.

## The situation

`cross_val_score(model, X, y, cv=5)` on a classifier uses stratified folds by default, but `cv=KFold(5)` does not, and neither does a hand-written split. On the admissions table sorted by the target, `KFold(5)` gave validation folds with 0%, 0%, 0%, 49% and 100% admitted.

```
cross_val_score(logit, X_sorted, y_sorted, cv=KFold(5), scoring="roc_auc")
# [ nan nan nan 0.954 nan ]      three folds have one class only: ROC-AUC
    is undefined
cross_val_score(logit, X, y, cv=StratifiedKFold(5, shuffle=True, random_state
    =0), scoring="roc_auc")
# [0.946 0.952 0.943 0.952 0.956]
```

- ▶ A nan is the loud version. The quiet version is a fold with 10% positives next to one with 50%: the scores swing, the mean means nothing, and you tune to noise.
- ▶ For regression use `KFold(5, shuffle=True, random_state=0)`: shuffling is what protects you from a sorted table; stratification only exists for classes.

# Mistake: Trusting Impurity Importance of a Noisy Column

## The situation

You add a column of random numbers to the admissions table. The forest's `feature_importances_` ranks it **8th of 56**, above the student's final grade. A team would keep it and drop the grade.

- ▶ Impurity importance counts how much the splits on a column reduced impurity *on the training rows*. A continuous column offers thousands of cuts and one of them always separates a few training rows by chance: columns with many distinct values (ids, noise) are inflated, two-valued columns deflated.
- ▶ Permutation importance on validation rows gave the noise column 0.0002 and the grade 0.0015: same forest, honest answer.
- ▶ Second trap: two copies of `math` each got half of its importance (0.056 and 0.055). Correlated columns share credit; drop one and the other's importance doubles.

## The rule

Decide what to keep with permutation importance or with the CV score after dropping, never with `feature_importances_` alone.

## The situation

PCA keeps the directions of largest spread. Nothing says those are the directions that separate the classes. On the admissions table, PCA to 10 components before logistic regression drops the CV ROC-AUC from **0.950 to 0.879**; to 5 components, 0.735.

| ECG, 10-tree forest | PCA(4) | PCA(8) | PCA(16) | 4 hand-made features |
|---------------------|--------|--------|---------|----------------------|
| balanced accuracy   | 0.303  | 0.352  | 0.768   | 0.46 to 0.97         |

- ▶ Four principal components of a heartbeat score *below* four summary statistics: the spread of a signal is dominated by its baseline and amplitude, not by the small wave that marks a disorder.
- ▶ Use PCA to look, to decorrelate for KNN, or to compress when the time limit bites. When the budget is “four numbers”, spend them on numbers that carry the label.

## The situation

The leaderboard you see during the contest is scored on a part of the test set (JOAI 2025: half of it). The final ranking uses the other part. Every upload is a peek at the public half, and twenty uploads with small changes are a model fitted to it.

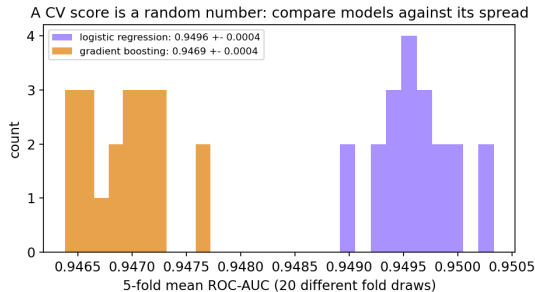
- ▶ JOAI 2025: the gold write-up's model scored 0.984 public and 0.981 private and *rose* about 30 places at the end, because teams above it had tuned to the public half and fell.
- ▶ The cross-validated score on 6,885 training rows is a steadier estimate than a public leaderboard on 320 test rows. When the two disagree, trust the one with more rows.
- ▶ Submission limits exist for this reason. Spend uploads on different ideas, not on the same idea with a different seed.

## The rule

Pick the final submission by CV before looking at the board. Then look, and change your mind only for a bug.

# Model Evaluation: **Tips**

# Tip: Report the Spread, Then Decide



Source: admissions data, 5-fold mean ROC-AUC repeated for 20 fold draws per model;  
scripts/day13.evaluation-figures.py.

- ▶ A CV mean moves when the folds move. Twenty draws: logistic  $0.9496 \pm 0.0004$ , boosting  $0.9469 \pm 0.0004$ .
- ▶ The gap (0.0027) is seven spreads wide, so it is real and the simpler model wins. A gap of 0.0005 would not have been.
- ▶ `scores.std()` from one `cross_val_score` gives the spread *between folds*, a coarser but free version of the same check. Print it next to every mean.

On a 300-row table the spread is ten times larger and most “improvements” vanish inside it; that is when repeated CV (several `random_state` values) earns its time.

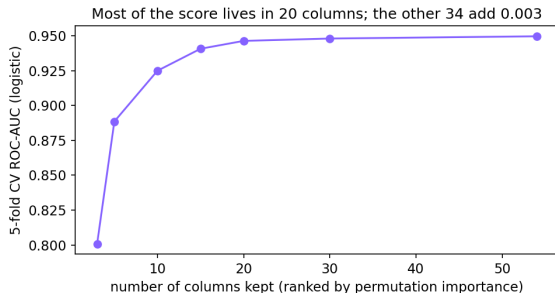
## Why

Cross-validation trains on 80% of the rows to measure; the submission should come from a model that saw 100%. The learning curve says what the last 20% is worth: the forest's validation ROC-AUC still rose from 0.932 to 0.935 between 4,000 and 5,508 rows.

```
from sklearn.base import clone
scores = cross_val_score(best, X, y, cv=cv, scoring="roc_auc")      # measure
    with the folds
final = clone(best).fit(X, y)                                       # then fit
    a fresh copy on all rows
test_pred = final.predict_proba(X_test)[: , 1]
```

- ▶ clone gives an unfitted copy with the same dials; the HAIO sample solution does exactly this before its submission.
- ▶ The alternative that also works: keep the five fold models and average their test predictions (the JOAI recipe). Same data used, plus a free blend.
- ▶ What *not* to do: submit the model from the last fold, which saw 80% of the rows and was never chosen for a reason.

# Tip: Drop Columns Only If the Score Agrees



Source: admissions data, columns ranked by permutation importance, logistic regression on the top  $k$ ; scripts/day13.evaluation\_figures.py.

- ▶ Top 5 columns: 0.889. Top 20: 0.946. All 54: 0.950. The last 34 columns are worth 0.003, which is real (seven spreads) but small.
- ▶ Keep all 54 for the score; keep 20 when a rule limits the columns, when the model must be explained, or when the time limit bites.

```
rank = pd.Series(permutation_importances_mean,
                 index=X.columns).
                 sort_values(
                     ascending=False)
for k in [5, 10, 20, 54]:
    cols = list(rank.index[:k])
    print(k, cross_val_score(logit, X[
        cols], y,
        cv=cv, scoring="roc_auc").
        mean())
```

# Tip: Write the Rules Into the Notebook

## Why

The Polish ECG task allowed at most 10 trees of depth at most 10 and at most 4 features; the grader clamps or disqualifies anything else. The official write-up asserts the limits in code so that no later edit can break them quietly. Every task has such lines: a submission limit, a column count, allowed libraries, a time limit.

```
N_TREES, MAX_DEPTH, N_FEATURES = 10, 10, 4
assert N_TREES <= 10 and MAX_DEPTH <= 10
features = make_features(X_train)
assert features.shape[1] == N_FEATURES, features.shape
rf = RandomForestClassifier(n_estimators=N_TREES, max_depth=MAX_DEPTH,
    random_state=42)
```

- ▶ Add `random_state` everywhere a model or a split is random (forests, boosting, `train_test_split`, `StratifiedKFold(shuffle=True)`). A score you cannot reproduce cannot be compared with the next one.
- ▶ Time the cells that matter (`time.time()` before and after `fit`): a model that takes 40 seconds to fit once takes 200 seconds in a 5-fold CV and 2,400 in a 12-cell grid.

# Model Evaluation: **Common Practices**

# Practice: The Submission Sanity Check

```
sub = pd.read_csv("submission.csv"); sample = pd.read_csv("sample_submission.csv")
assert sub.shape == sample.shape and list(sub.columns) == list(sample.columns)
# Day 06: the sample's shape
assert (sub["ID"].values == test["ID"].values).all()
# same rows, same order
print(sub["admitted"].describe())
# range, mean, no NaN
print("predicted positive rate", (sub["admitted"] >= 0.5).mean(), "train rate",
      , y.mean()) # 0.27 vs 0.30
```

- ▶ The HAIO sample solution prints the submission shape and compares the predicted admission rate with the training rate before saving; the Day 11 bug (a mean prediction of −633,806 dollars) dies at the describe line.
- ▶ For a ROC-AUC task the column must hold probabilities: `sub["admitted"].nunique() > 2`. The HAIO sample solution itself saves `predict()` labels in its last cell; do not copy that line.
- ▶ For regression clip to the training range (`clip(lower=0)` for prices) and check for NaN: one NaN can score the whole file zero.

# Practice: One Notebook Shape for Every Tabular Task

```
# 1 load + EDA (Day 06 routine)
train and test
def make_features(d): ...
X, y, X_test = make_features(train), train[TARGET], make_features(test).reindex(columns
    =..., fill_value=0)
# 3 folds that mirror the test
saved
cv = StratifiedKfold(5, shuffle=True, random_state=0)
oof, test_pred = {}, {}
for name, model in models.items():
    oof[name] = cross_val_predict(model, X, y, cv=cv, method="predict_proba")[:, 1]
    test_pred[name] = clone(model).fit(X, y).predict_proba(X_test)[:, 1]
    print(name, roc_auc_score(y, oof[name]))
# 5 threshold / blend chosen on oof
# 4 models, each scored the same way, OOF
# 6 write, check, upload
```

- ▶ Saving oof and test\_pred per model makes blending and threshold choice a one-line experiment later, with no refitting.
- ▶ The first run uses the baseline model only and ends with an upload (Day 06's fifteen-minute rule); every later run adds one model or one column.
- ▶ A notebook with this shape can be restarted and run top to bottom; that is also how the organisers check it when they ask for the code.

## The loop the Polish ECG write-up followed

1. Version 1: four features (how rough the signal is outside the main spike; the height and the position of the largest deviation; the mean level after the spike), forest of 10 trees, `class_weight="balanced"`: balanced accuracy **82.97%**, 35 points.
  2. The confusion matrix showed PVC beats recalled at 58% and 47 of them called normal. A narrower version of the position feature made it *worse*: 80.84%.
  3. Version 3 changed the preprocessing, not the features: remove the slow drift of the baseline, smooth with a 5-point moving average before looking for the spike. Same four features, same forest: **97.37%**, 97 points.
- ▶ Fourteen points of balanced accuracy and 62 points of score came from reading a  $5 \times 5$  matrix and asking “which class is missed, and what would a doctor look at?”.
  - ▶ With the organisers’ own four statistics (min, max, mean, std) the same forest scores 0.462; with `class_weight="balanced"` 0.551. The per-class table tells you where the next ten points are; the overall number does not.

# Model Evaluation: **Olympiad Practice**

## Task

The ECG task of Day 10: 2,000 training beats of 150 voltages, 1,500 validation beats, five classes (1,400 of the training beats are normal). At most 4 features computed with NumPy or SciPy, a RandomForestClassifier with at most 10 trees of depth at most 10, balanced accuracy on a hidden test set: 0 points at 0.75 or below, 100 at 0.98. A team has the organisers' baseline (min, max, mean, std: balanced accuracy 0.462) and one afternoon. Plan the afternoon as an evaluation problem: what do they measure, on which rows, in which order, and when do they stop?

- ▶ **Question 1:** the task ships a separate validation set. Should the team also cross-validate, and if so with which splitter?
- ▶ **Question 2:** after each change, which single table do they print, and what in it decides the next change?
- ▶ **Question 3:** the forest is fixed at 10 trees. Name two settings they are still free to change and one they must not touch.

Adapted from II Olimpiada AI (Poland, 2025), stage 1, task 3, and its official write-up (versions 1 to 3 of the authors' solution).

1. **Measure:** StratifiedKFold(5, shuffle=True) on the 2,000 training beats *and* the given validation set (five imbalanced classes, so stratify); two numbers that agree beat one. Never the training accuracy: a depth-10 forest memorises.
2. **Print after every change:** the  $5 \times 5$  confusion matrix and the per-class recall; balanced accuracy is their mean, so the lowest recall is where the next feature goes.
3. **Free:** class\_weight="balanced" (0.462 to 0.551 with the baseline features), the preprocessing before the features (the write-up's 83% to 97%), a fixed random\_state.  
**Not free:** the feature count, the trees, the depth; assert them.

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
rf = RandomForestClassifier(n_estimators=10, max_depth=10, class_weight="balanced",
    random_state=42)
F_train, F_val = features(X_train), features(X_val); assert F_train.shape[1] <= 4
oof = cross_val_predict(rf, F_train, y_train, cv=cv)
pred = rf.fit(F_train, y_train).predict(F_val)
print(balanced_accuracy_score(y_train, oof), balanced_accuracy_score(y_val, pred))
print(confusion_matrix(y_val, pred), recall_score(y_val, pred, average=None).round(2))
```

**Stop** when the two balanced accuracies agree and sit above the band you need; the hidden test set lands near them, not near the training score.

# Model Evaluation: **Check Yourself**

Answer before turning the page.

1. **Definition.** A training table has 2,000 rows from 400 patients, five visits each, and the test rows come from 100 patients nobody has seen. Which splitter do you use, and what would a random KFold score tell you?
2. **Mechanism.** Model A scores  $0.912 \pm 0.004$  and model B  $0.915 \pm 0.004$  over 20 fold draws. Is B better? What would make you sure either way?
3. **Code.** What is wrong here, and which score will it inflate?

```
rf = RandomForestClassifier(300, random_state=0).fit(X_train, y_train)
imp = pd.Series(rf.feature_importances_, index=X_train.columns)
keep = imp.sort_values(ascending=False).index[:15]
print(cross_val_score(rf, X_train[keep], y_train, cv=5, scoring="roc_auc").
      mean())
```

1. GroupKFold with the patient id as the group (every patient entirely on one side). A random KFold would let the model see the same patient's other visits and report the score for *recognising* patients, which the test set of new patients cannot reward; on the credit table the gap was 0.754 against 0.689.
2. The gap is 0.003 with spreads of 0.004: inside the noise, so you cannot say. Repeating with more fold draws (or more folds) shrinks the spread; if the gap stays and the spread falls below about a third of it, B is better. Otherwise take the simpler or faster model.
3. Two problems. The columns were chosen by impurity importance, which rewards noisy high-cardinality columns; and the choice used the same rows the CV then scores, so columns that fit the training rows by chance stay in and the CV score is optimistic. Select with permutation importance, inside each fold or on a separate split.

# Model Evaluation: **Limitations & What Comes Next**

- ▶ **No scheme fixes a test set that differs from the training set.** If the judge's rows come from another year or another hospital, every fold is optimistic; only the task statement can tell you, so read it.
- ▶ **Cross-validation costs time.** Five folds of a 40-second model is three minutes; a grid on top is an hour. Subsample for the search, cross-validate the finalists only.
- ▶ **Out-of-fold tuning is still tuning.** Thirty thresholds and ten blends tried on the OOF table consume it a little; keep the decisions few and simple.
- ▶ **Permutation importance measures a model, not the world.** A column the model ignored can still matter; a column it leaned on can be a proxy for another.
- ▶ **PCA is linear.** It cannot unroll a curved pattern; Week 5's embeddings do that job.

## The common mistake

Running the whole routine once, at the end. It is a loop: every new column or model goes through steps 3 to 5 again.

- ▶ **Tomorrow:** the week's Coursera exam and the mock competition. Everything in this deck applies; the routine frame is the plan.
- ▶ **Week 3:** neural networks. The validation habits stay exactly the same; the new dials (learning rate, epochs, early stopping) will look like boosting's.

# Model Evaluation: **Summary**

- ▶ **Folds that mirror the test:** stratified and shuffled for classes, shuffled for regression, grouped when rows share an id; check whether the test shares the groups.
- ▶ **Out-of-fold predictions** are your private test set: choose thresholds, blends and columns on them, never on the test file or the public board.
- ▶ **The spread decides:** a gap smaller than the spread is not a result; repeated CV shrinks the spread.
- ▶ **PCA** shows a 150-column table in two dimensions and compresses it; it does not know the labels and usually lowers a supervised score.
- ▶ **Importance:** permutation on validation rows, not impurity on training rows; prune only when the CV score agrees.
- ▶ **The routine:** metric, baseline, folds, OOF, compare, refit on all rows, write, check, upload, stop trusting the board.

# Model Evaluation: **References & Resources**

- [1] Hungarian AI Olympiad (2025). *Online qualifier: University Admissions*, data, starter notebook and sample solution (CC BY-NC-SA 4.0). The fold, noise, OOF, learning-curve and importance figures; the labels-for-AUC mistake. <https://github.com/Hungarian-AI-Olympiad/HAI0-Hungarian-AI-Olympiad/tree/main/2025/nyari-online>
- [2] Olimpiada Nationala de Inteligenta Artificiala, Romania (2025). *Simulare OJIA 1: Credit Score*; data and solved notebook in <https://github.com/jaredliw/ioai-tsp-2025>. The grouped-rows experiment.
- [3] Olimpiada AI (2025). *II Olimpiada Sztucznej Inteligencji, etap 1, zadanie 3: Wykrywanie zaburzeń sygnału EKG*, data and official write-up (versions 1 to 3). The PCA figure, the class-weight numbers and the practice task. [https://github.com/OlimpiadaAI/II-OlimpiadaAI/tree/main/1\\_etap](https://github.com/OlimpiadaAI/II-OlimpiadaAI/tree/main/1_etap)
- [4] rayramy04 (2025). *JOAI 2025 solutions* (stratified folds, fold-model averaging, public against private leaderboard). <https://github.com/rayramy04/joai2025-solutions>
- [5] Sandei, S. (2025). *roai-solved*, NEOAI 2025 Tricy Table solution (single hold-out, refit on all rows). <https://github.com/stefanasandei/roai-solved/tree/main/international-contests/neoai/2025>
- [6] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. StratifiedKFold, GroupKFold, cross\_val\_predict, learning\_curve, permutation\_importance, PCA. <https://scikit-learn.org>

[7] All plots: `slides-builder/scripts/day13_evaluation_figures.py` (matplotlib, real data only).